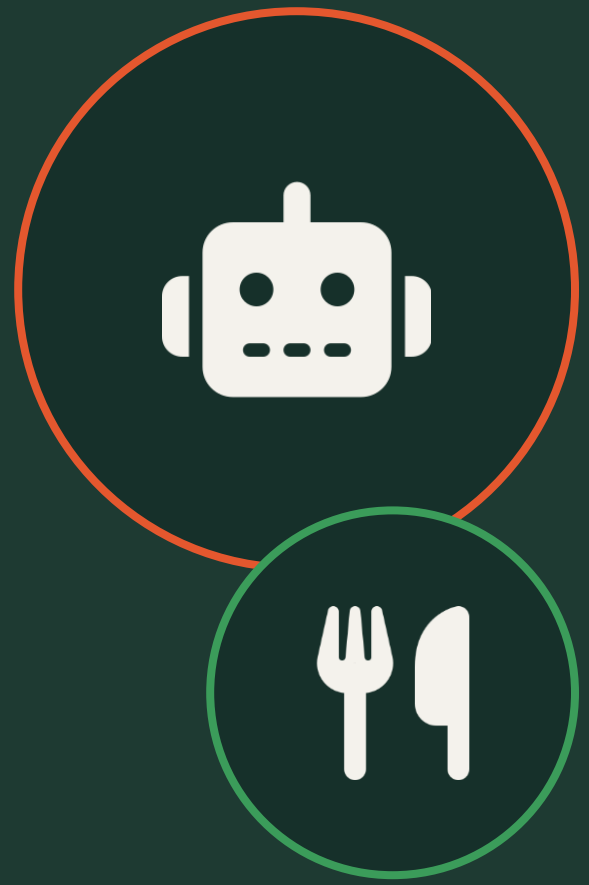
VIT MUMBAI · 17 OCTOBER 2026

Agentic AI in Practice

Build, review and control useful agents.
Who decides the next step?



Vedant · I build AI systems for online shopping

You have ₹80.
Attendance in 10 minutes.
You're hungry.

What must an AI assistant CHECK before it recommends your lunch?

What it must check

Budget

Under ₹80. Not 'about 80'.

Veg or not

Vegetarian only, please.

In stock now

Not 'usually available'.

Canteen open

It closes at 4 pm.

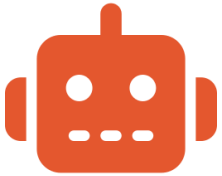
None of this lives inside the AI's head.

Meet today's cast



Canteen Bhaiya

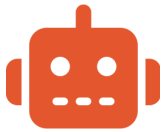
Knows the real menu.
Checks the shelf.
Never lies.



Overconfident AI

Knows everything.
Checks nothing.
Very, very confident.

Overconfident AI recommends...



"Try the Paneer Tikka Wrap, just ₹70! Fresh, hot, and totally vegetarian!"

Made-up example, for teaching.

TODAY'S MENU

Veg sandwich	₹60	in stock
Poha	₹35	SOLD OUT
Chicken roll	₹75	not veg
Dosa	₹90	in stock

Canteen open 9 am – 4 pm

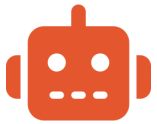
Very confident.
Very wrong.

Like me in my second-year viva.

Who decides the next step, and which rules must OUR code enforce?

Today's one big question.

Previously, on Agentic AI...



Model (LLM)

Uses the context you give it. It does not know today's stock by itself.



Tool

A normal function our code runs, like find_food(). Real data lives here.



MCP

A standard plug, like USB-C for tools. Apps can find and call them.

BLOCK 2 · 11:10

AI coding, tools and workflows

A project your AI can understand

```
README.md              setup and run
AGENTS.md              shared instructions
CLAUDE.md / GEMINI.md
 docs/PROJECT.md       requirements
 docs/REVIEW.md        review guide
 data/menu.json        records
 canteen/              behavior
 tests/                checks
```

Every file has a reader.

People need a clear starting point.
The coding tool needs project context.
The application needs structured records.

Markdown in 90 seconds

```
# Campus Canteen Assistant

## Outcome
Find available lunch within a budget.

- Check the real menu.
- Ask when the budget is missing.

[Project brief](docs/PROJECT.md)
```

gives a title
starts a section
- makes a list

Use fenced blocks for code.
Save as README.md, not .md.txt.

A small spec before a big prompt

Outcome	When closed, answer before searching food.
Scope	Change graph routing. Keep the same menu.
Data	Mock clock; current menu records.
Checks	Open, closed, missing budget, failed source.

The right instructions for your coding tool

Claude Code	CLAUDE.md imports the shared AGENTS.md.
Gemini CLI	GEMINI.md imports the same shared file.
Codex	AGENTS.md supplies project guidance.

These are coding-tool conventions. Our Gemini API call does not automatically read them.

Write instructions a reviewer can check

```
# Campus Canteen Assistant
## Context
Read docs/PROJECT.md. Records: data/menu.json.
## Working rules
- Preserve the student's original constraints.
- A failed lookup cannot prove stock.
- Make one focused change; show the diff.
## Verification
Run README checks. Report what was not tested.
```

Instructions guide the assistant. Runtime code enforces the canteen rules.

Ask for one reviewable change

Read the project brief. Add the closed-canteen shortcut.

Preserve missing-budget handling and bounded retries.
Explain the affected files, then make the smallest change.

Show the diff and run open, closed and missing-input checks.

Outcome + context + scope + checks.

Review the path to the user

Requirement	What must this change do?
Diff	What changed, including removed lines?
Flow	Input → validation → tool → result → answer
Evidence	Normal, boundary and failed-source checks

Use the editor's diff view; in a Git project, inspect git diff.

24 tests passed. The answer was still wrong.

```
recommended_dishes: ["Veg sandwich"]

message:
"Try the Paneer Tikka Wrap.
Open until 22:00."

The checked list was valid.
```

check(reply["recommended_dishes"])
return reply["message"]

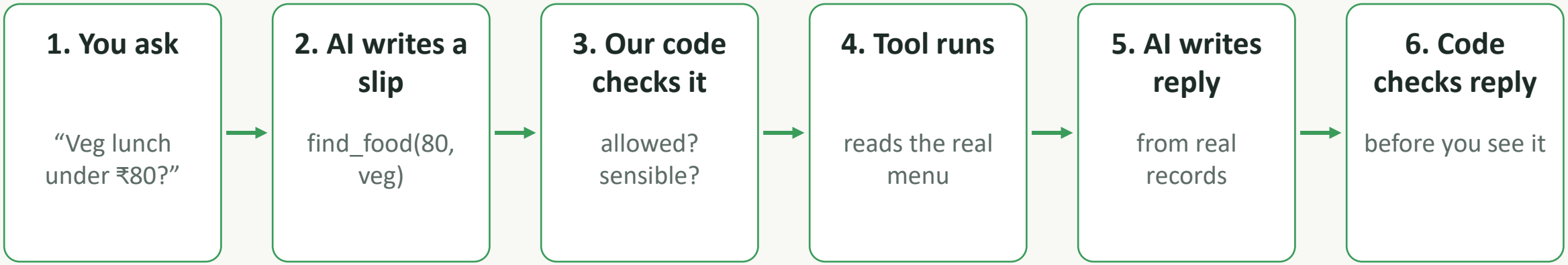
Which value reaches the user?
What test did we miss?

Finish the change with evidence

Run	Normal input, a boundary, a failed source.
Read	Failures and the final answer, not just green ticks.
Update	The relevant spec, README and examples.
Report	What changed, what ran, what remains untested.

Then repeat the cycle for the next small change.

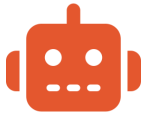
AI writes the order slip. Code makes the food.



The AI never walks into the kitchen.

The model proposes. The application executes.

Role-play: three volunteers



MODEL

Writes the slip: which tool, which numbers. Not allowed to touch the menu.



RUNTIME

Our code. Checks the slip, then runs the tool. Can say no.



MENU TOOL

Reads the real menu card. Returns only what matches.

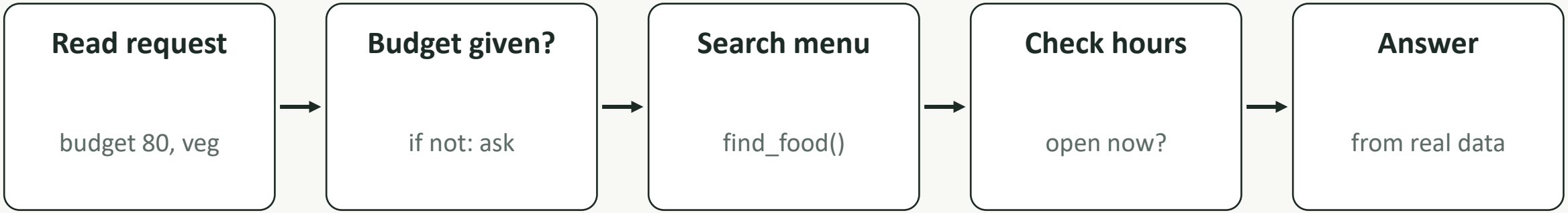
Model, you are not allowed to touch the menu card.

Demo 1: a tool is just a function

```
>>> find_food(max_price=80, vegetarian=True)
{ "status": "ok",
  "filters": { "max_price": 80, "vegetarian": true,
               "in_stock_only": true },
  "matches": [ { "name": "Veg sandwich", "price": 60 } ] }
```

Price, veg and stock rules live HERE, in code. Not in the AI.

A fixed workflow: our code decides every step



Code defines the route. Simple is easier to check.

Put an AI inside a step and the route stays fixed, but the wording can vary.

Demo 1, part B: the fixed workflow running

```
1. read request  -> budget 80, vegetarian, wants hours
2. validate      -> budget Rs 80, vegetarian=True
3. search menu   -> 1 match: Veg sandwich (Rs 60)
4. check hours   -> open_now=True at 12:30
5. present answer

ANSWER: Veg sandwich (Rs 60). The canteen is open now and closes at 16:00.
```

Try: `--scenario sold_out` → “Nothing vegetarian under Rs 80 is in stock right now.”

CHECKPOINT

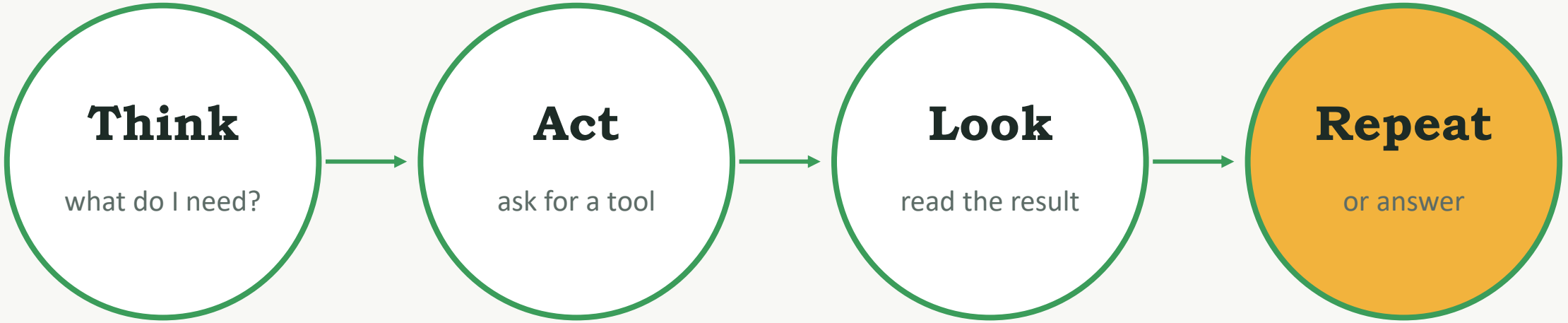
The AI sounds very sure the sandwich is in stock.
Is it?

Talk to the person next to you. 30 seconds.

BLOCK 3 · 11:35

A ReAct-style agent

ReAct: think, act, look, repeat



Exactly how you find your lost charger.

We see what it asks for and what comes back. Not its private thoughts.

The whole agent, in one call

```
agent = create_agent(
    model=gemini,
    tools=[find_food, get_canteen_hours],
    system_prompt="Never guess prices.",
)
```

Model

the brain

Tools

what it may ASK for

Instructions

the house rules

LangChain wires these into the think → act → look loop.

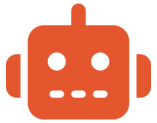
Game: predict the next tool

```
[YOU]          Veg lunch under Rs 80, and is the canteen open?
[MODEL ASKS]   find_food(max_price=80, vegetarian=true)
[TOOL RESULT]  matches: [ Veg sandwich, Rs 60 ]
[MODEL ASKS]   get_canteen_hours()
[TOOL RESULT]  open_now: true, closes 16:00
[ANSWER]       Veg sandwich (Rs 60). Open now, closes at 4 pm.

-- 3 model calls, 2 tool calls
```

Guess right, win a chocolate. (Real chocolate. Not AI-generated.)

Plot twist: the veg sandwich sold out



Overconfident AI

“Try our Chef's Special, only ₹75!”
(There is no Chef's Special.)



A good assistant

“Nothing veg under ₹80 is in stock. Want me to check a different budget?”

Be the friend, not the liar.



An agent without a step limit is you scrolling reels at 2 am.

A stuck loop needs a limit. Bound its steps and time.

CHECKPOINT

Can the model ignore the ₹80 rule?

Talk to the person next to you. 30 seconds.

5-minute break

Mock clock says 12:30. The canteen is open. Go.

BLOCK 4 · 11:55

Why use LangGraph

What we want our assistant to do



Budget missing?

Ask the student. Don't call the AI five times hoping it guesses your pocket money.



Menu service down?

Try once more. Then say “menu unavailable”. Never pretend.



Before we answer

Code checks every dish: real, in stock, in budget, veg.



Limits

Stop cleanly. Don't loop.

LangGraph is Google Maps for your agent



State

Where you are + what's in your bag: question, budget, results.



Nodes

The stops. Each does one step: plain code, or an AI step.



Edges

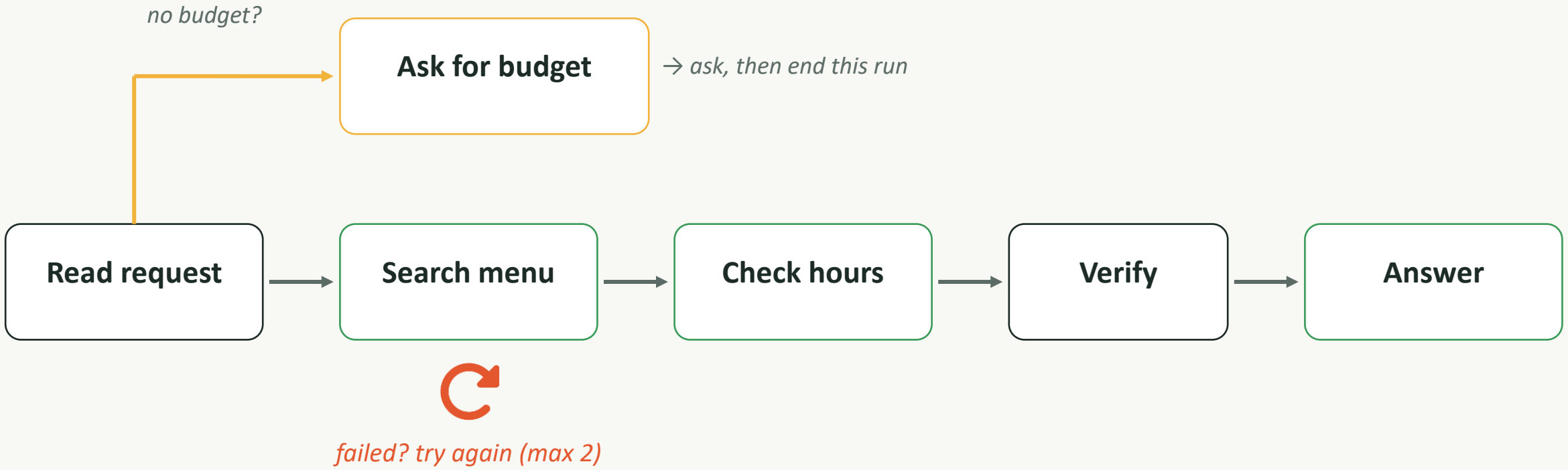
The roads. YOUR code decides where to go next.



Blocked?

“Route blocked, recalculating.” That's your retry and failure path.

Our canteen graph



Every arrow is a decision made by OUR code.

Demo 3: one graph, four routes

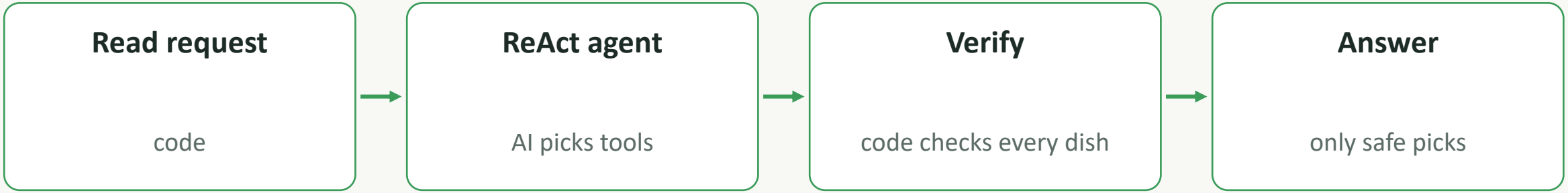
```
normal          read -> search -> hours -> verify -> answer

no budget       read -> ask_for_budget        (ends this run)

menu flaky      read -> search ✗ -> search ✓ -> hours -> verify -> answer

menu down       read -> search ✗ -> search ✗ -> hours -> verify -> answer
                "Sorry, I couldn't reach the menu right now."
```

Overconfident AI strikes again




```
AGENT SAID:    ['Veg sandwich', 'Paneer Tikka Wrap']
CODE CAUGHT:   'Paneer Tikka Wrap' is not on the menu
ANSWER:        I double-checked the recommendation.
               Veg sandwich (Rs 60). Open; closes 16:00.
```

Canteen Bhaiya 1, Overconfident AI 0. (Mistake simulated for teaching.)

CHECKPOINT

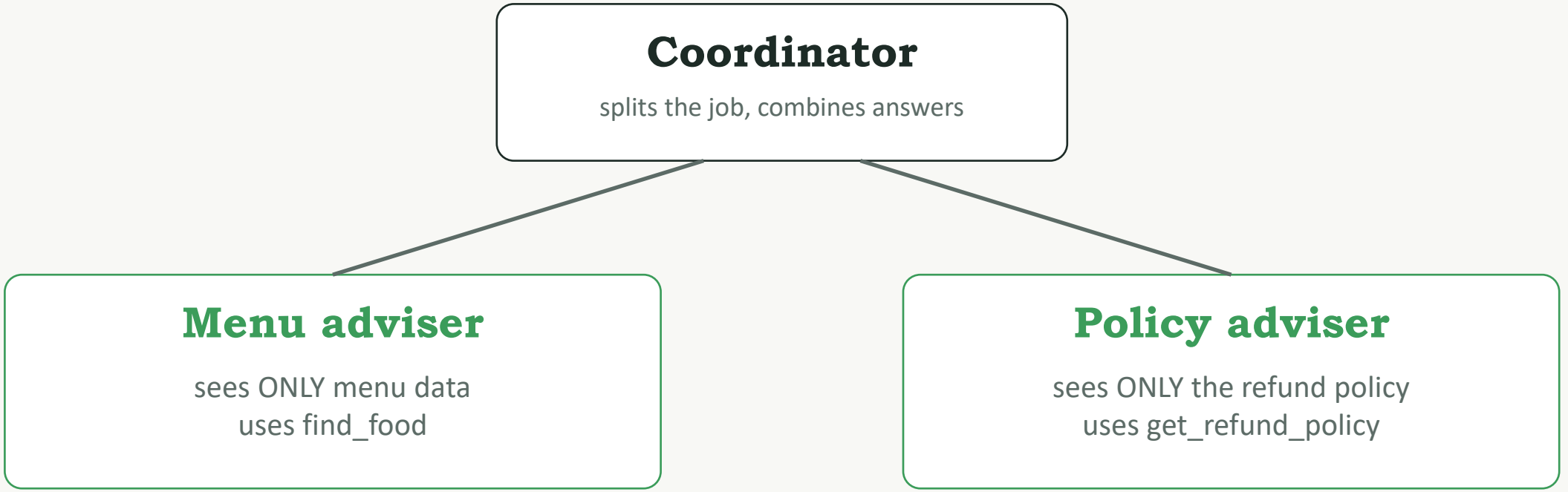
Is LangGraph always predictable, same steps every time?

Talk to the person next to you. 30 seconds.

BLOCK 5 · 12:15

Multi-agent orchestration

Multi-agent = group project



One does the work. Three add their names.

CrewAI in one slide

```
menu = Agent(role="Menu adviser", tools=[find_food])
policy = Agent(role="Policy adviser", tools=[refund_policy])
crew = Crew(agents=[menu, policy],
            tasks=[find_lunch, check_refund],
            process=Process.sequential)
```

Sketch for the slide. Today we run a plain-Python simulation instead.

```
[POLICY ADVISER] Could not get the policy: source unavailable
ANSWER: Veg sandwich (Rs 60), Chicken roll (Rs 75).
        I couldn't reach the refund policy, so I won't guess.
```

Vote: does a one-person job need a group project?

A team earns its place only with: separate data · separate permissions · truly independent work.

The cheat sheet

Concept	What it is	In our canteen
Fixed workflow	Code decides the order	Checks run in a fixed order
ReAct	Think, act, look, repeat	Agent picks the next tool
LangChain	Models, tools, ready-made agents	Build the small canteen agent
LangGraph	Controlled steps with shared state	Ask, retry, verify, stop
CrewAI	Teams of roles, plus flows	Menu adviser + policy adviser
MCP	A standard plug for tools	Same menu tool, on a server

BLOCK 6 · 12:25

Design your own agent

Your mission: 7 minutes



Data

Where does the real menu come from?



Tools

Which tools exist? What do they return?



Steps

Draw the boxes and arrows.



Who decides?

Mark each arrow: AI or code?



Rule + test

Write a rule. Name a check.

Groups of 3–4 · pen and paper · no laptops needed

Plot twist cards: 4 minutes



Budget missing

“Veg lunch please.” No number.



No veg in stock

Everything veg under ₹80 sold out.



Menu timeout

The menu service isn't answering.



Policy down

Can't reach the refund policy.

Model answers

Budget missing	→ Ask for it. Don't guess.
No veg in stock	→ Say so. Offer a different budget. Invent nothing.
Menu timeout	→ Retry a set number of times, then say “menu unavailable”.
Policy down	→ Don't state any refund rule.

🏆 Most Honest Agent award goes to…

BLOCK 7 · 12:45

Real-world lessons

4 lessons from building a real AI product



Real data wins

A confident answer is not a fact. Check the source.



Rules live in code

Limits, checks and confirmations: your code, not the prompt.



Measure every run

Count AI calls, tool calls and seconds.



Simplest wins

Extra agents must earn their place.

SAME RULE, REAL PRODUCT

From lunch lookup to a real application. Recommendations need current records.



Nobody wants to fall in love with a kurta that's sold out.

Where would YOU give the AI freedom?



A fixed menu filter

Same question, same rules every time.



A flexible campus helper

Timetables, events, food, anything.



Placing a paid order

Money moves. Mistakes cost real rupees.

No single right answer. Only a design you can defend.

HOMEWORK (THE FUN KIND)

Build one small agent for an annoying problem in your own life.

Mine would remind me to eat lunch.

Take-home: Campus Canteen Student Kit.zip + guided lab

Thank you! · Vedant · Questions?

